Step 1: Update Restaurant Model

Open:

```
api/models.py
```

Update model:

```
from django.db import models


class Restaurant(models.Model):
    name = models.CharField(max_length=100)
    cuisine = models.CharField(max_length=100)
    location = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Step 2: Create ModelSerializer

Open:

```
api/serializers.py
```

Add:

```
from rest_framework import serializers
from .models import Restaurant


class RestaurantSerializer(serializers.ModelSerializer):
```

```
    class Meta:
        model = Restaurant
        fields = '__all__'
```

Step 3: Install django-filter

Run:

```
pip install django-filter
```

Step 4: Add Apps in settings.py

Open:

```
foodiehub/settings.py
```

Add:

```
INSTALLED_APPS = [

    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'rest_framework',
    'django_filters',

    'api',
]
```

Step 5: Configure REST Framework Settings

Inside:

```
foodiehub/settings.py
```

Add:

```
REST_FRAMEWORK = {

    # Page Number Pagination
    'DEFAULT_PAGINATION_CLASS':
        'rest_framework.pagination.PageNumberPagination',

    'PAGE_SIZE': 3,

    # Filter Backend
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.OrderingFilter',
    ]
}
```

This will:

- Show only 3 restaurants per page
- Enable filtering
- Enable ordering

Step 6: Create RestaurantViewSet

Open:

```
api/views.py
```

Add:

```
from rest_framework import viewsets
from rest_framework.pagination import (
    PageNumberPagination,
```

```
    LimitOffsetPagination
)

from .models import Restaurant
from .serializers import RestaurantSerializer


# -------------------------------
# Page Number Pagination
# -------------------------------

class RestaurantPagination(PageNumberPagination):
    page_size = 3


# -------------------------------
# Limit Offset Pagination
# -------------------------------

class RestaurantLimitOffsetPagination(
    LimitOffsetPagination
):
    default_limit = 2


# -------------------------------
# Restaurant ViewSet
# -------------------------------

class RestaurantViewSet(viewsets.ModelViewSet):

    queryset = Restaurant.objects.all()
    serializer_class = RestaurantSerializer

    # Pagination
    pagination_class = RestaurantPagination

    # Ordering
    ordering_fields = ['name', 'cuisine']

    # Filtering
    filterset_fields = ['cuisine']
```

Step 7: Register ViewSet with DefaultRouter

Open:

```
api/urls.py
```

Add:

```
from rest_framework.routers import DefaultRouter
from .views import RestaurantViewSet

router = DefaultRouter()

router.register(
    'restaurants',
    RestaurantViewSet,
    basename='restaurants'
)

urlpatterns = router.urls
```

Step 8: Include API URLs

Open:

```
foodiehub/urls.py
```

Add:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

Automatically Generated Endpoints

DRF Router automatically creates:

```
GET      /api/restaurants/
POST     /api/restaurants/
GET      /api/restaurants/1/
PUT      /api/restaurants/1/
PATCH    /api/restaurants/1/
DELETE   /api/restaurants/1/
```

Step 9: Test PageNumberPagination

Create more than 3 restaurants.

Example POST:

```
{
    "name": "Pizza Hub",
    "cuisine": "Italian",
    "location": "Mumbai"
}
```

Now open:

```
http://127.0.0.1:8000/api/restaurants/
```

You will get:

```
{
    "count": 5,
    "next": "http://127.0.0.1:8000/api/restaurants/?page=2",
    "previous": null,
    "results": [
        ...
    ]
}
```

Only 3 restaurants appear per page.

Step 10: Switch to LimitOffsetPagination

Open:

```
api/views.py
```

Replace:

```
pagination_class = RestaurantPagination
```

With:

```
pagination_class = RestaurantLimitOffsetPagination
```

Now test:

```
http://127.0.0.1:8000/api/restaurants/?limit=2&offset=2
```

Meaning:

- Skip first 2 records
- Return next 2 records

Example:

If database contains:

```
1. Pizza Hub
2. Burger Point
3. Spice Garden
4. Sushi House
```

Response will return:

```
Spice Garden
Sushi House
```

Step 11: Test Ordering

Order by Name

```
http://127.0.0.1:8000/api/restaurants/?ordering=name
```

Reverse Order by Cuisine

```
http://127.0.0.1:8000/api/restaurants/?ordering=-cuisine
```

`-` means descending order.

Step 12: Test Filtering

Create restaurants with different cuisines.

Example:

```
{
    "name": "Pizza Hub",
    "cuisine": "Italian",
    "location": "Delhi"
}
```

```
{
    "name": "Spice Garden",
    "cuisine": "Indian",
    "location": "Ahmedabad"
}
```

Now filter:

```
http://127.0.0.1:8000/api/restaurants/?cuisine=Italian
```

Only Italian restaurants will appear.
